Chapter 6: Deep Neural Networks – Advanced Architectures for Chemical Applications

2. Recurrent Neural Networks (RNN)

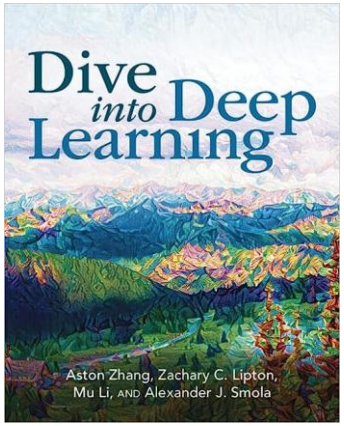
Chem 361

Xuhui Huang

Outline

- **Recurrent neural networks (RNNs)**
  - Forward prorogation
  - Backpropagation
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - Long short-term memory (LSTM) model
- Optimizing RNN training for better predictions
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

Reading material:



Chapter 9

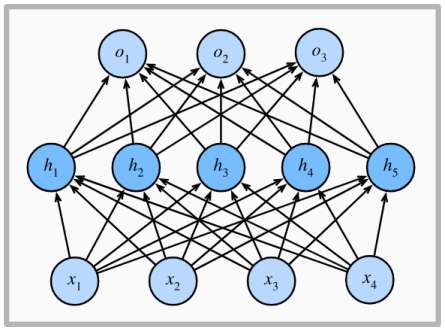
https://d2l.ai/

© Xuhui Huang

Tailoring Neural Network Architectures to Chemical Data Structures

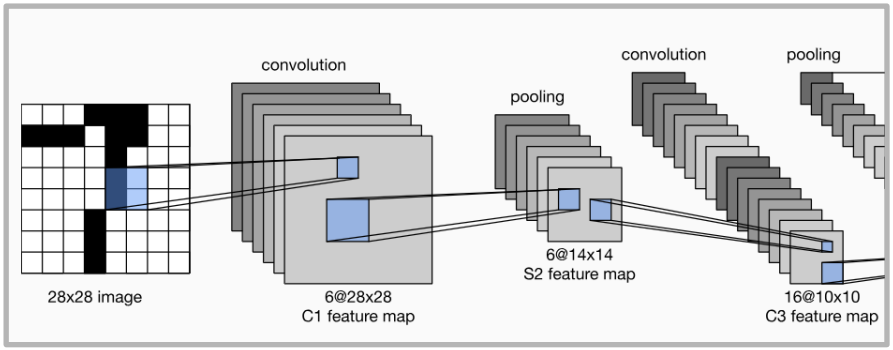
- Data type: **Tabular features**

MLP & other Feed forward NN (FNN)



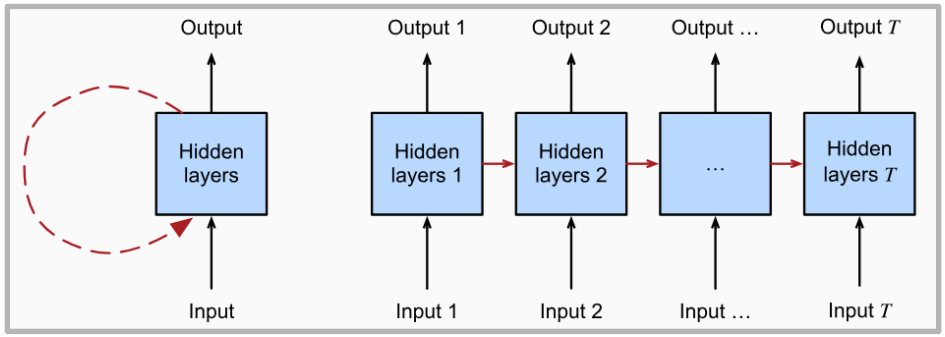
- Data type: **Grids/images**

Convolutional neural network (CNN)



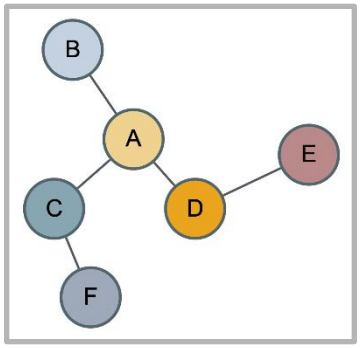
- Data type: **Sequential data**

Recurrent neural networks (RNNs)



- Data type: **Graphs**

Graph neural networks (GNNs)



https://d2l.ai/

© Xuhui Huang

Sequential Data

Language:

Previously, when dealing with individual inputs, we assumed that they were sampled independently from the same underlying distribution $P(X)$. While we still assume that entire sequences (e.g., entire documents or patient trajectories) are sampled independently, we cannot assume that the data arriving at each time step are independent of each other. For example, the words that likely to appear later in a document depend heavily on words occurring earlier in the document. The medicine a patient is likely to receive on the 10th day of a hospital visit depends heavily on what transpired in the previous nine days.

https://d2l.ai/

Financial Times Stock Exchange 100 Index

Predict tomorrow's stock price using historical price data.

https://google.com/

© Xuhui Huang

Sequential Chemical Data

Sequential data:

Language, videos, music …

SMILES strings as sequential chemical data:

Vocabulary: {'C': 1, 'O': 2, '=': 3, 'N': 4, 'pad': 0}

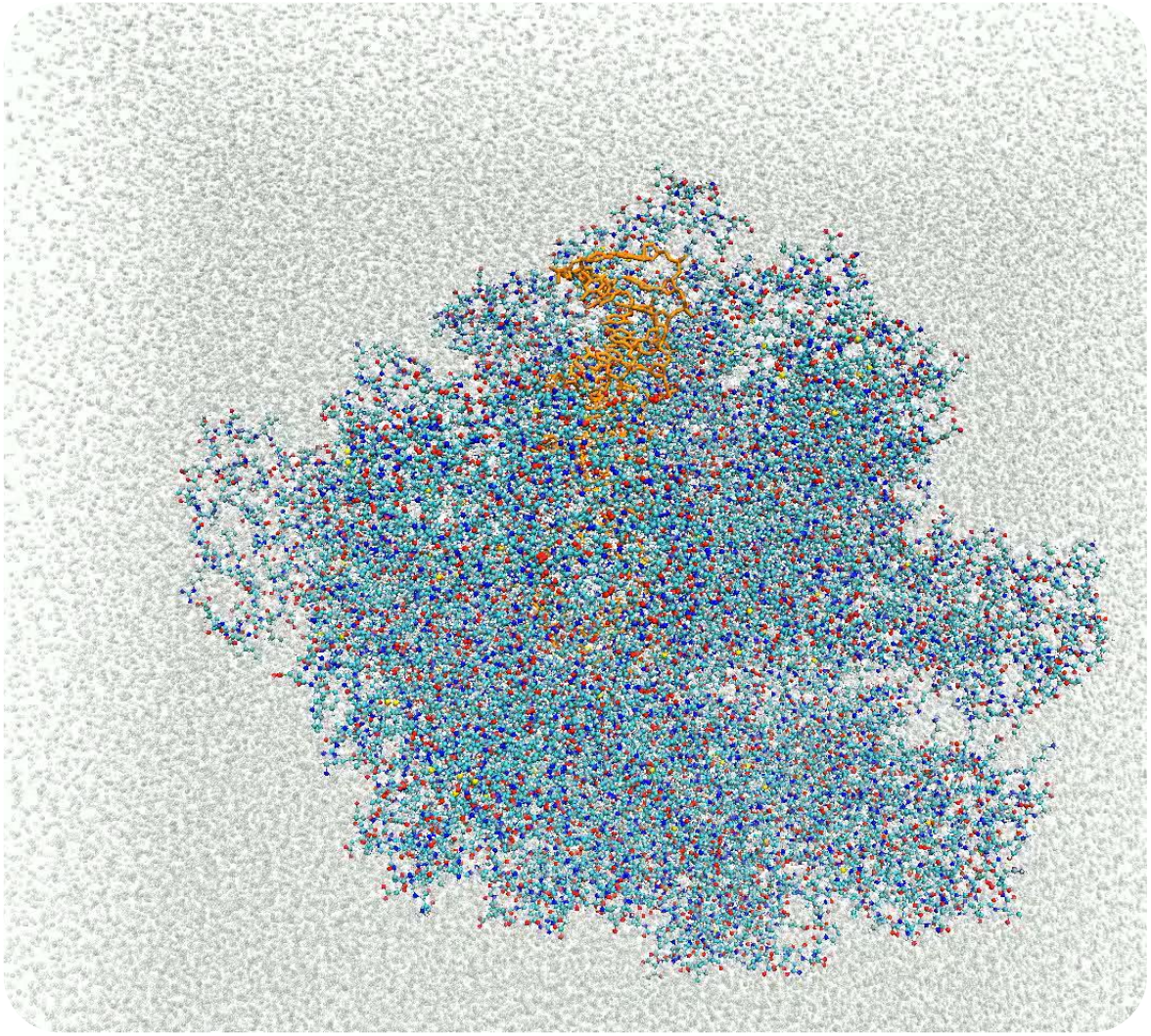
Pad sequences to a fixed length of 4.

Ethanol (CCO): [1, 1, 2]	**[1, 1, 2, 0]**
Formaldehyde (C=O): [1, 3, 2]	**[1, 3, 2, 0]**
Acetaldehyde (CC=O): [1, 1, 3, 2]	**[1, 1, 3, 0]**
Ethanolamine (OCCN): [2, 1, 1, 4]	**[2, 1, 1, 4]**

Sequential data!

© Xuhui Huang

Sequential Chemical Data



1 day on an A100 GPU - 80ns (80 million steps)

RNAP, >500K atoms

Molecular Dynamics (MD) Simulations: Time sequence data.

© Xuhui Huang

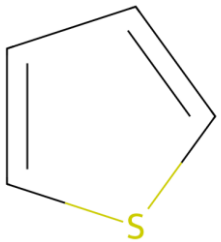
Outline

- Recurrent neural networks (RNNs)
  - **Forward prorogation**
  - Backpropagation
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - Long short-term memory (LSTM) model
- Improving RNN prediction
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

© Xuhui Huang

Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.



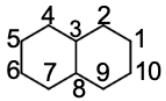
c1cccs1

Tokenize:

['c', '1', 'c', 'c', 'c', 's', '1']

Tokens are the atomic (indivisible) units of text.

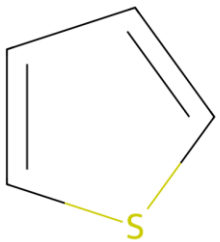
SMILES strings

	Rules	**Example**
Atoms	Element Symbol Ions Using brackets	C, N, Cl, … Proton: [H+] Implicit hydrogens can be omitted
Bonds	Single bond: "-" Double bond: "=" Triple bond: "#" Aromatic bond: ":" or lower case (Single bond and aromatic bond are often omitted)	Benzene: c1ccccc1 or C1=CC=CC=C1 Hydrogen cyanide: C#N
Branches	Put branch in parenthesis after the atom it connects to	Acetic acid: CC(=O)O
Rings	Put the same indices after atoms connected in the ring but not adjacent in the string	 C1CC2CCCCC2CC1

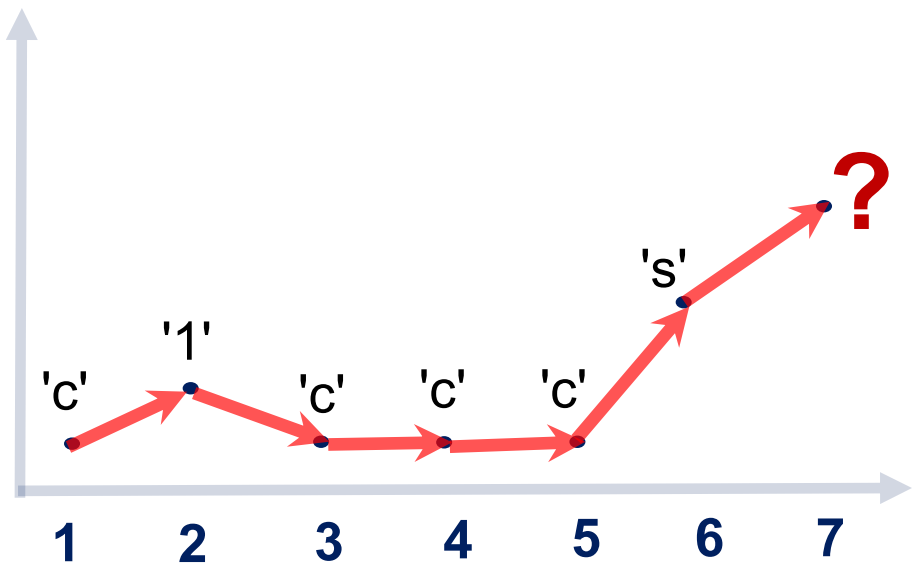
© Xuhui Huang

Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.



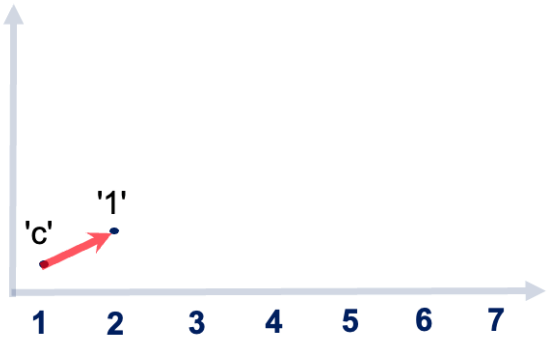
c1cccs1



© Xuhui Huang

Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.



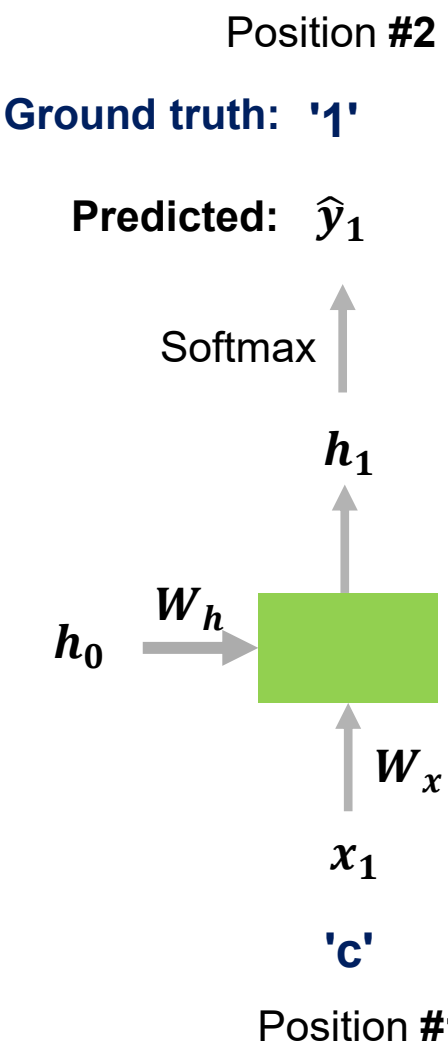
Position #1 to Position #2.

Linear combination:

$$z_1 = W_x x_1 + W_h h_0$$

Activation:

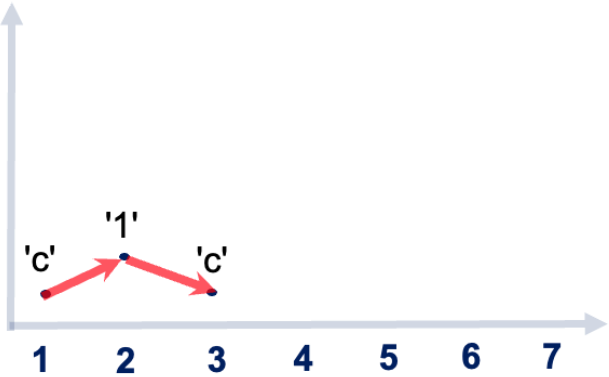
$$h_1 = \tanh(z_1)$$



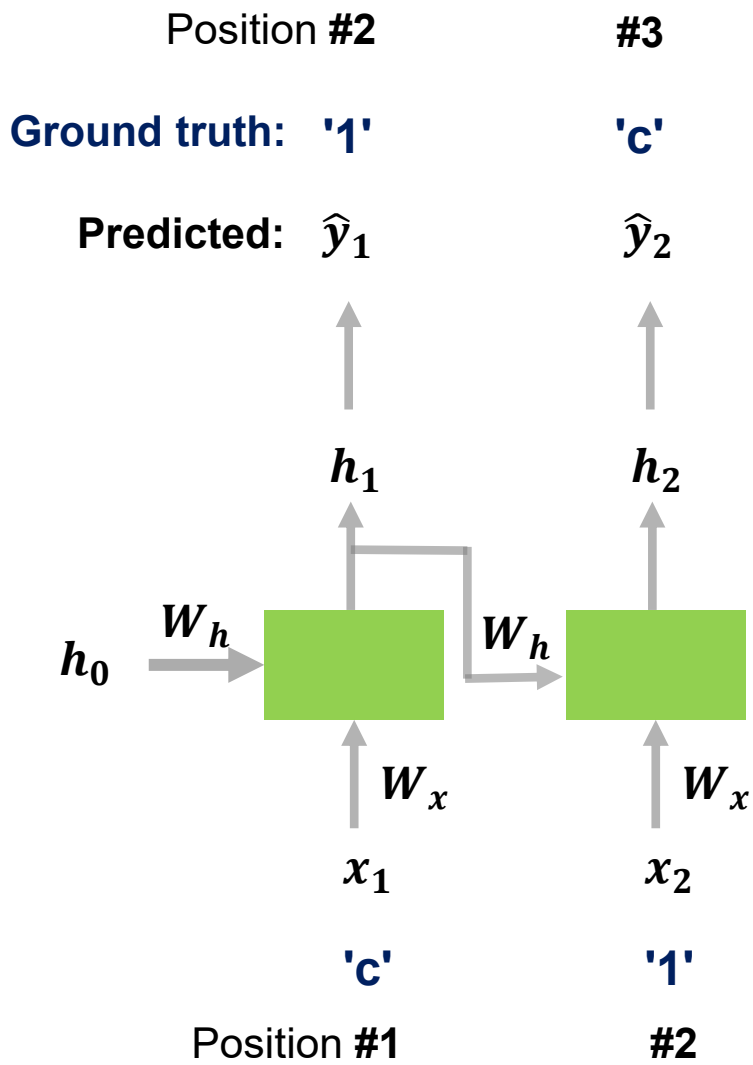
© Xuhui Huang

Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.



Position #1, #2 to Position #3.



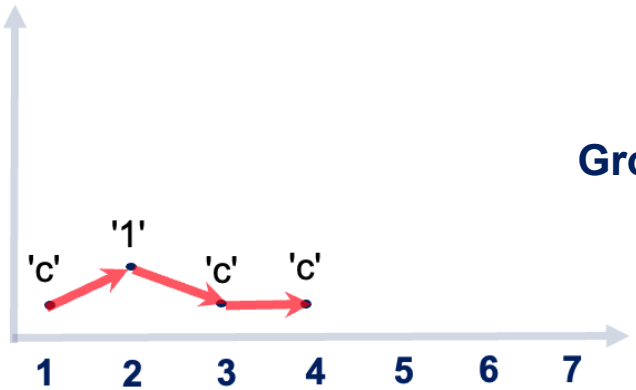
$$h_2 = \tanh(W_x x_2 + W_h h_1)$$

W_h and W_x do not change.

© Xuhui Huang

Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.

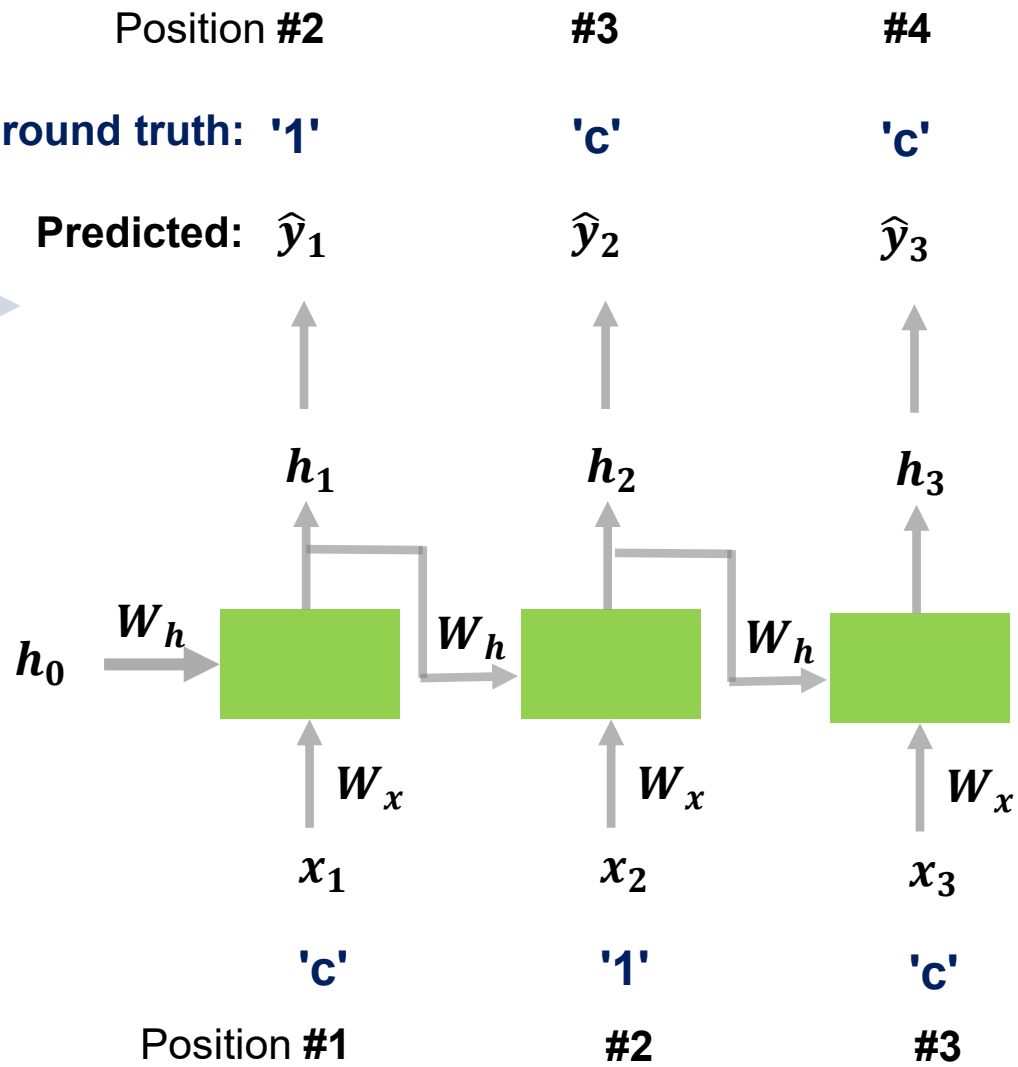


Hidden state:

$$h_3 = \tanh(W_x x_3 + W_h h_2)$$

- $\hat{y}_3$ is based on x_1, x_2 and x_3, because h_2 retains history information of x_1 and x_2

In RNN, the hidden state of the current cell retains the history information!



© Xuhui Huang

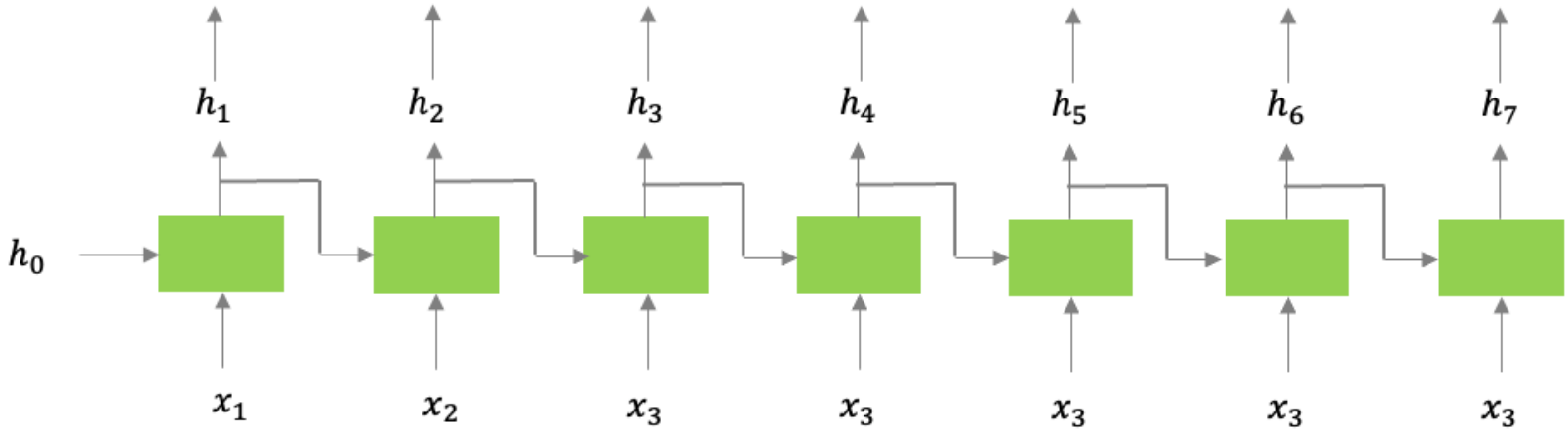
Recurrent Neural Networks (RNNs)

Task: Predicting the next token of SMILES strings of chemical compounds.

Hidden states in RNNs share the same weights.

EOS: End of Sentence

Loss	$l(y_1, \hat{y}_1)$	$l(y_2, \hat{y}_2)$	$l(y_3, \hat{y}_3)$	$l(y_4, \hat{y}_4)$	$l(y_5, \hat{y}_5)$	$l(y_6, \hat{y}_6)$	$l(y_7, \hat{y}_7)$
Truth	'1'	'c'	'c'	's'	'c'	'1'	$< EOS >$
Prediction	$\hat{y}_1$	$\hat{y}_2$	$\hat{y}_3$	$\hat{y}_4$	$\hat{y}_5$	$\hat{y}_6$	$\hat{y}_7$



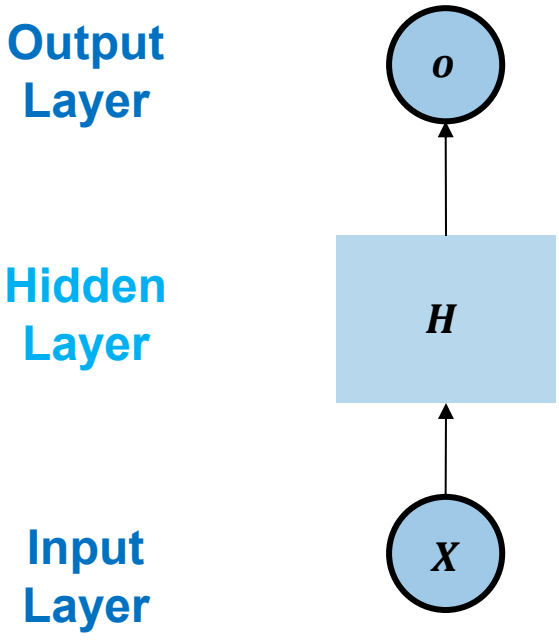
Total Loss: $\mathcal{L} = l(y_1, \hat{y}_1) + l(y_2, \hat{y}_2) + \cdots + l(y_t, \hat{y}_t) = \sum_{t=1}^{7} l(y_t, \hat{y}_t)$

© Xuhui Huang

Recurrent vs. Feed Forward NNs

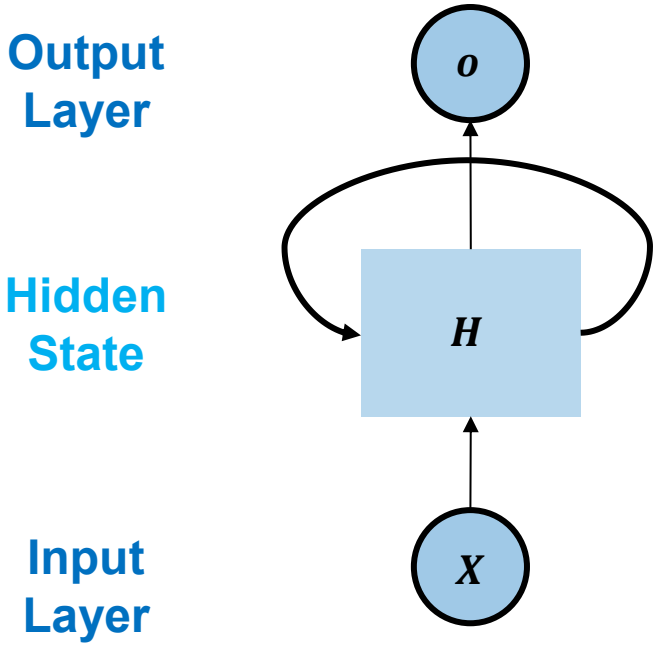
RNNs contain hidden states, which can only be computed by looking at data at previous time steps.

MLP



$$h = \sigma(Wx)$$

RNN



The layer gets its own "previous" value as inputs

$$h_i = \sigma(W_x x_{i-1} + W_h h_{i-1})$$

© Xuhui Huang

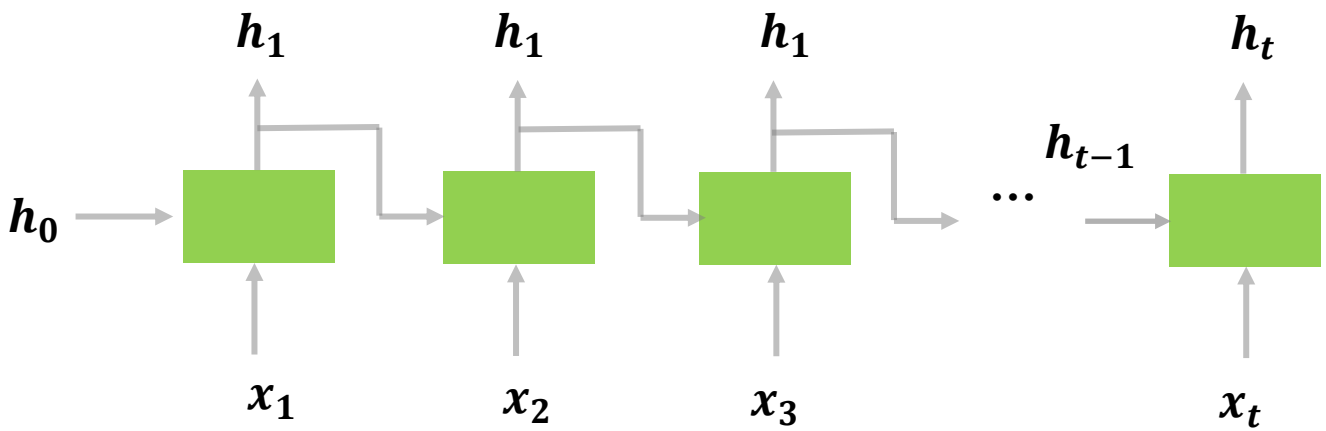
Outline

- Recurrent neural networks (RNNs)
  - Forward prorogation
  - **Backpropagation**
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - Long short-term memory (LSTM) model
- Optimizing RNN training for better predictions
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

© Xuhui Huang

Backpropagation Through Time (BPTT)

An **unrolled RNN** is a feedforward neural network with shared weights across time steps.



Backpropagation: Apply the chain rule to backpropagate gradients through the unrolled RNN.

$$\frac{\partial l(y_t, \hat{y}_t)}{\partial W_h} = \frac{\partial l(y_t, \hat{y}_t)}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial h_t} \frac{\partial h_t}{\partial W_h}$$

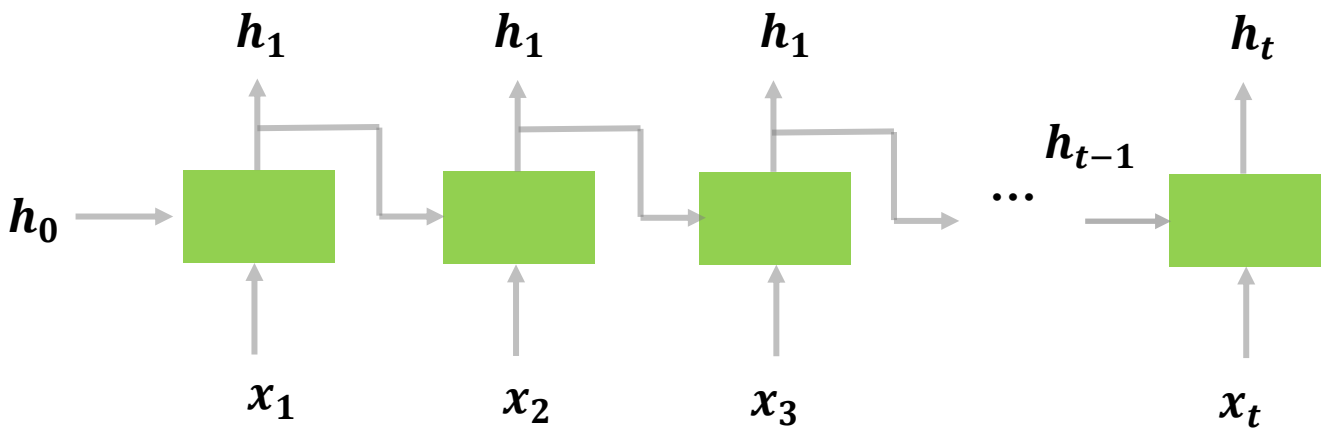
$$h_t = \tanh(W_x x_t + W_h h_{t-1}) = f_t(x_t, h_{t-1}, W_h)$$

$$\frac{\partial h_t}{\partial W_h} = \frac{\partial f_t}{\partial W_h} + \frac{\partial f_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_h} = \cdots = \frac{\partial f_t}{\partial W_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^{t} \frac{\partial f_j}{\partial h_{j-1}} \right) \frac{\partial f_i}{\partial W_h}$$

© Xuhui Huang

Backpropagation Through Time (BPTT)

An **unrolled RNN** is a feedforward neural network with shared weights across time steps.



Backpropagation: Apply the chain rule to backpropagate gradients through the unrolled RNN.

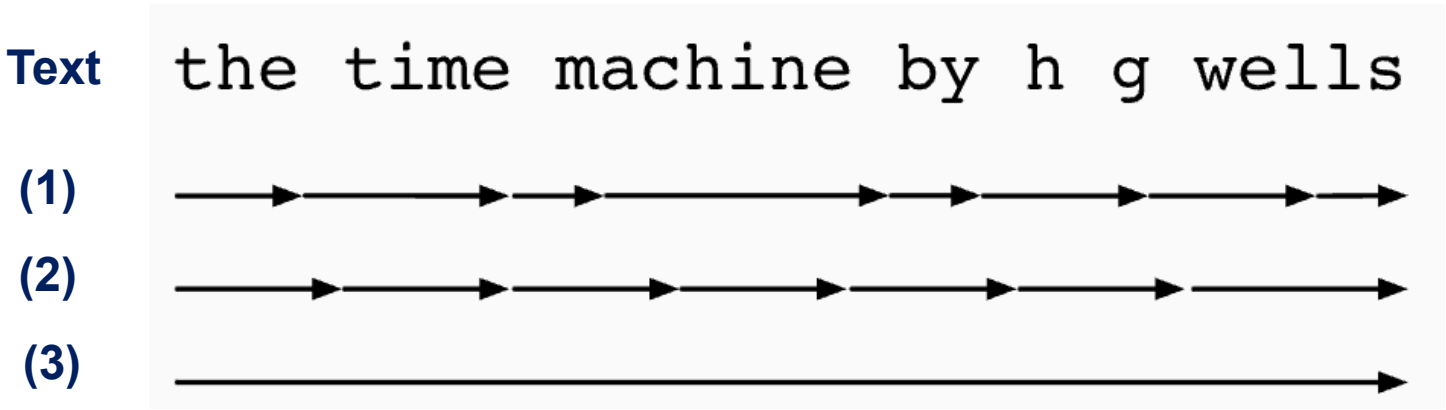
$$\frac{\partial h_t}{\partial W_h} = \frac{\partial f_t}{\partial W_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^{t} \frac{\partial f_j}{\partial h_{j-1}} \right) \frac{\partial f_i}{\partial W_h}$$

Computing the full sum is slow and prone to **gradient exploding or vanishing**, as small changes in initial conditions can significantly impact the outcome – the butterfly effect.

© Xuhui Huang

Backpropagation Through Time (BPTT)

Gradient computation **strategies** in RNN backpropagation.



(1). Randomized Truncation partitions the text into segments of varying lengths.

(2). Regular truncation breaks text into fixed-length subsequences –commonly used.

(3). Full BPTT computes gradients over the entire sequence, but impractical due to computational cost.

https://d2l.ai/

© Xuhui Huang

Outline

- Recurrent neural networks (RNNs)
  - Forward prorogation
  - Backpropagation
- **Gradient Vanishing/Exploding**
  - **Gradient Clipping**
  - Long short-term memory (LSTM) model
- Optimizing RNN training for better predictions
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

© Xuhui Huang

Clipping Gradient to Avoid Exploding

Gradient Clipping: Clip the gradients, forcing the resulting "clipped" gradients to take smaller values and prevent gradient exploding.

Gradient descent: $W_{t+1} \leftarrow W_t - \alpha \cdot \hat{\boldsymbol{g}}$

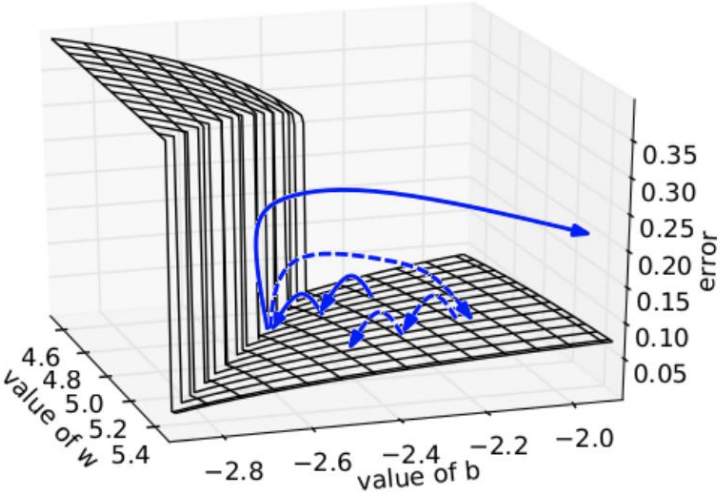
$\hat{\boldsymbol{g}} = \nabla_W \ell(\hat{y}, y)$: gradient. $\boldsymbol{\alpha}$: learning rate.

Clip the gradient: $$\boxed{\hat{\boldsymbol{g}} \leftarrow \min\left(1, \frac{\theta_0}{||\hat{\boldsymbol{g}}||}\right)\hat{\boldsymbol{g}}}$$

θ_0: threshold for gradient clipping.

- The gradient norm never exceeds θ_0.
- The updated gradient is aligned with the original direction of $\hat{\boldsymbol{g}}$.

Error of a single hidden state RNN



Solid lines: standard gradient descent.

Dashed lines: gradient clipping.

Pascanu, Mikolov, Bengio. (2012). DOI: 10.48550/ARXIV.1211.5063

© Xuhui Huang

Exercise

Which of the following statements about standard RNNs are true?

(A). Standard RNNs suffer from both vanishing and exploding gradient problems during training.

(B). Gradient vanishing in RNNs makes it difficult to learn long-term dependencies.

(C). Gradient clipping is a common technique used to mitigate the vanishing gradient problem in RNNs.

(D). Exploding gradients occur when gradients grow exponentially during backpropagation, leading to unstable training.

© Xuhui Huang

Exercise

Which of the following statements about standard RNNs are true?

(A). Standard RNNs suffer from both vanishing and exploding gradient problems during training. - True

(B). Gradient vanishing in RNNs makes it difficult to learn long-term dependencies. - True

(C). Gradient clipping is a common technique used to mitigate the vanishing gradient problem in RNNs. – False

Gradient clipping is used to address gradient exploding, not gradient vanishing.

(D). Exploding gradients occur when gradients grow exponentially during backpropagation, leading to unstable training. - True

© Xuhui Huang

Outline

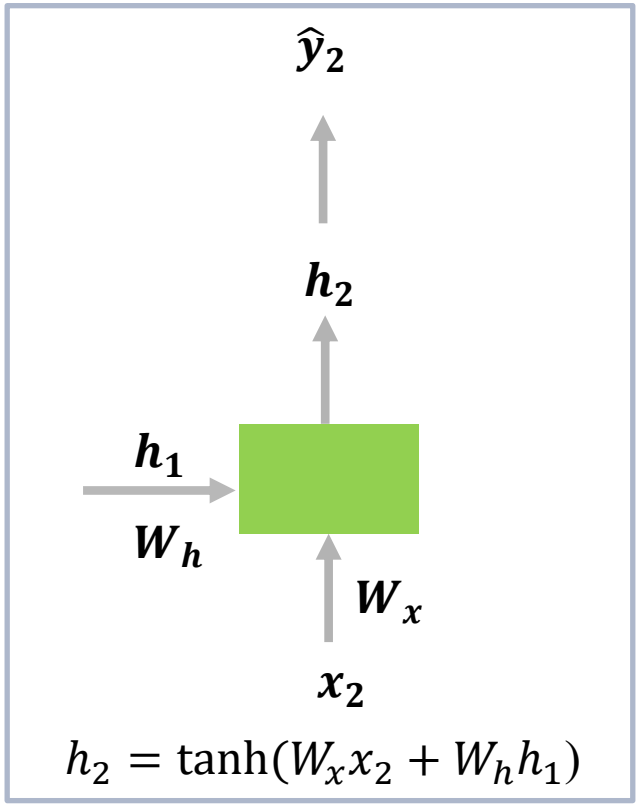
- Recurrent neural networks (RNNs)
  - Forward prorogation
  - Backpropagation
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - **Long short-term memory (LSTM) model**
- Optimizing RNN training for better predictions
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

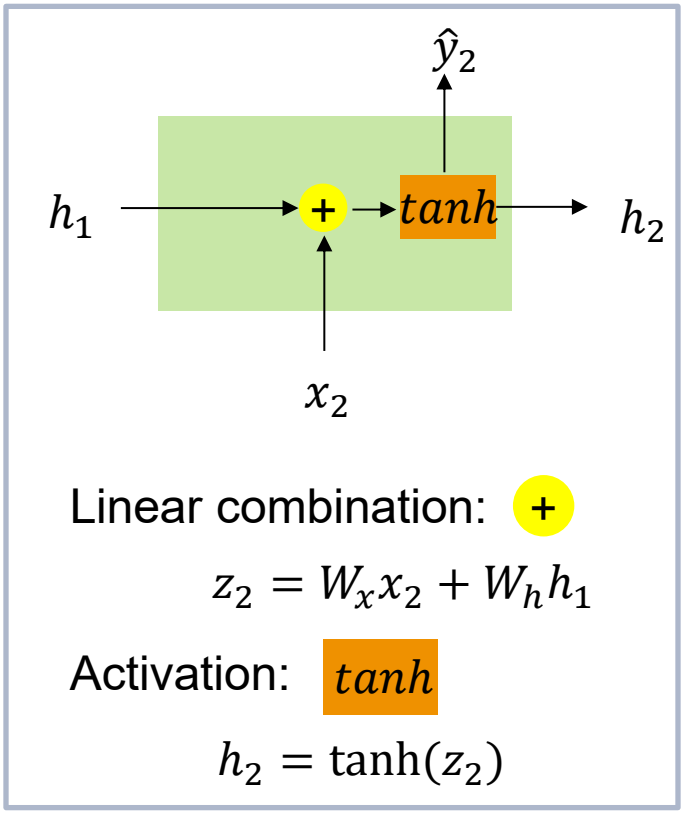
© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM extends standard RNNs by replacing each recurrent node with a memory cell. This cell has built-in memory that allows gradients to flow across many time steps without vanishing or exploding.

Representing a **standard RNN** as a CNN cell:



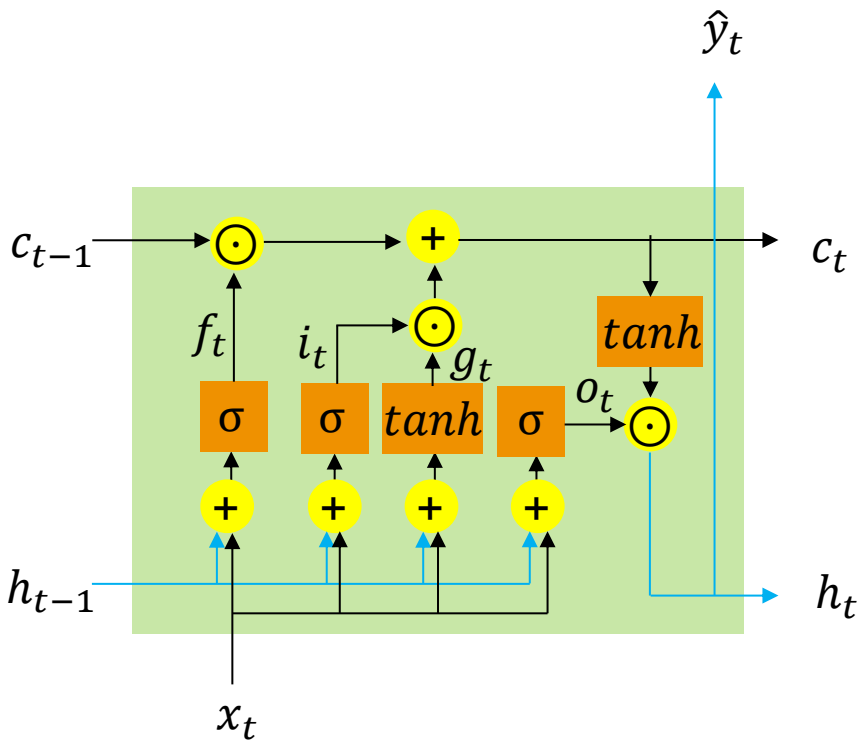


© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM extends standard RNNs by replacing each recurrent node with a memory cell. This cell has built-in memory that allows gradients to flow across many time steps without vanishing or exploding.

LSTM contains **a memory cell**:

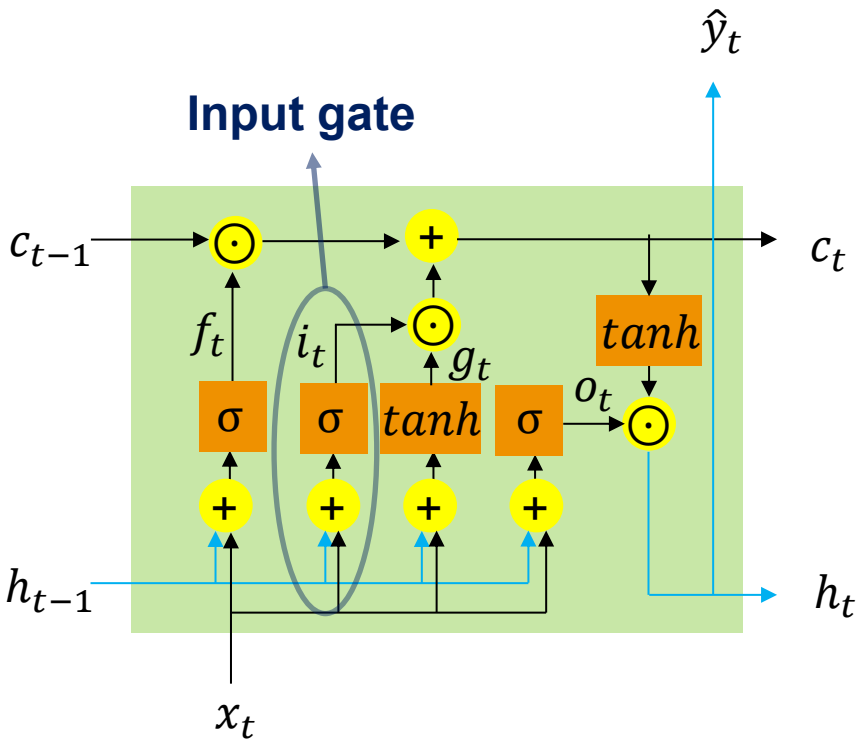


- Linear combination: +
- Activation with tanh: $tanh$
- Activation with sigmoid: σ
- Element-wise Multiplication: $\odot$

© Xuhui Huang

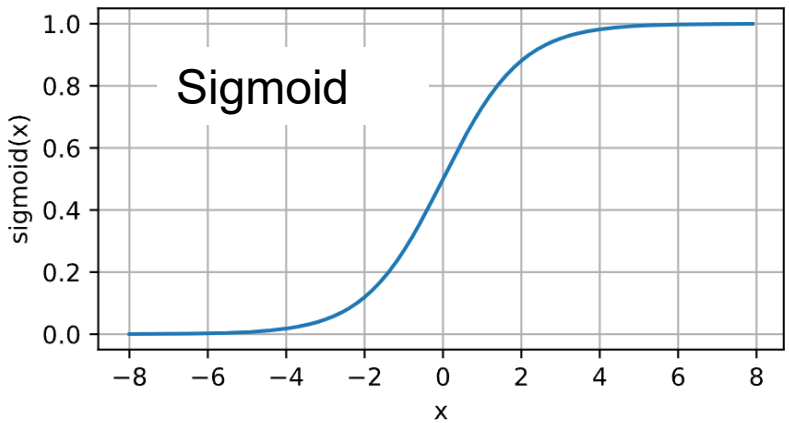
Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input gate controls how much of the input value (x_t) is added to the memory cell's internal state:

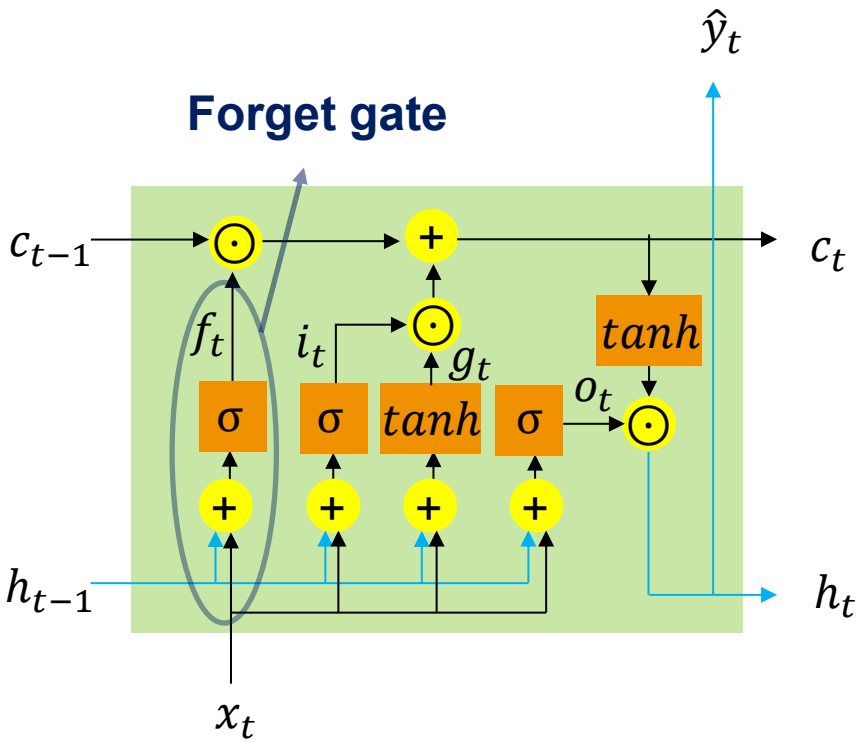
$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1})$$



© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input gate controls how much of the input value (x_t) is added to the memory cell's internal state:

$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1})$$

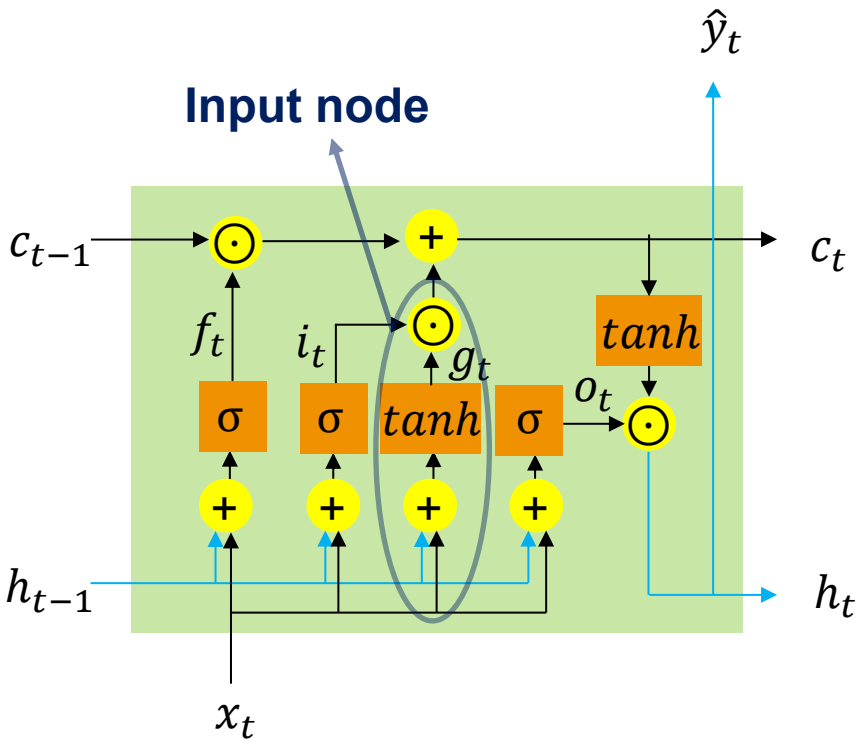
Forget gate controls whether to retain or discard the memory's current value

$$f_t = \sigma(W_{if}x_t + W_{hf}h_{t-1})$$

© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input gate controls how much of the input value (x_t) is added to the memory cell's internal state:

$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1})$$

Forget gate controls whether to retain or discard the memory's current value

$$f_t = \sigma(W_{if}x_t + W_{hf}h_{t-1})$$

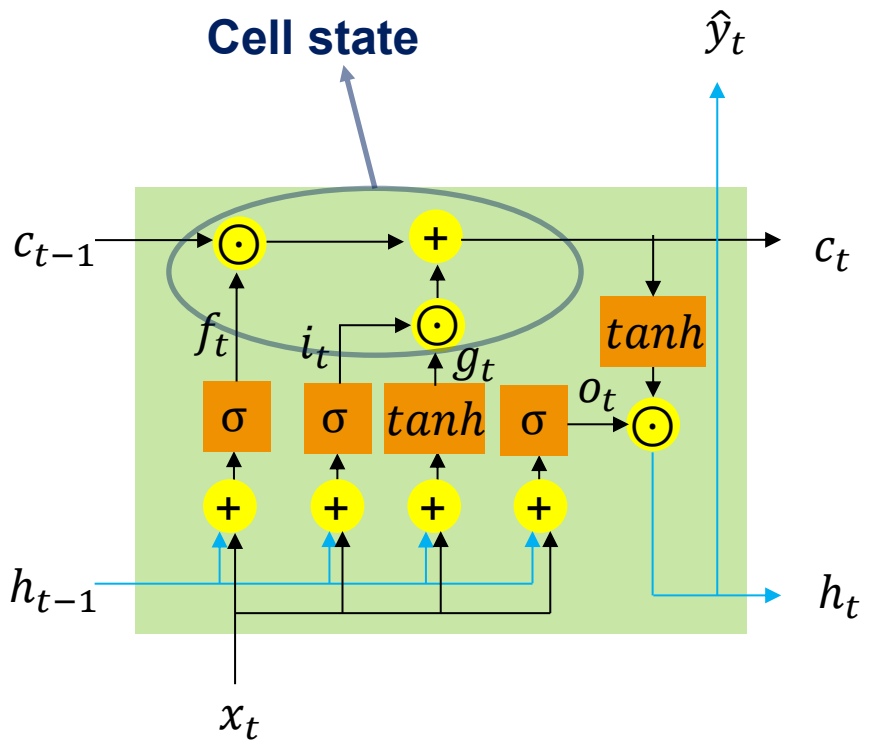
Input node – normal input as in standard RNN:

$$g_t = tanh(W_{ig}x_t + W_{hg}h_{t-1})$$

© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input gate controls how much of the input value (x_t) is added to the memory cell's internal state:

$$i_t = \sigma(W_{ii}x_t + W_{hi}h_{t-1})$$

Forget gate controls whether to retain or discard the memory's current value

$$f_t = \sigma(W_{if}x_t + W_{hf}h_{t-1})$$

Input is modified by the **Input** and **Forget** gates

Cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

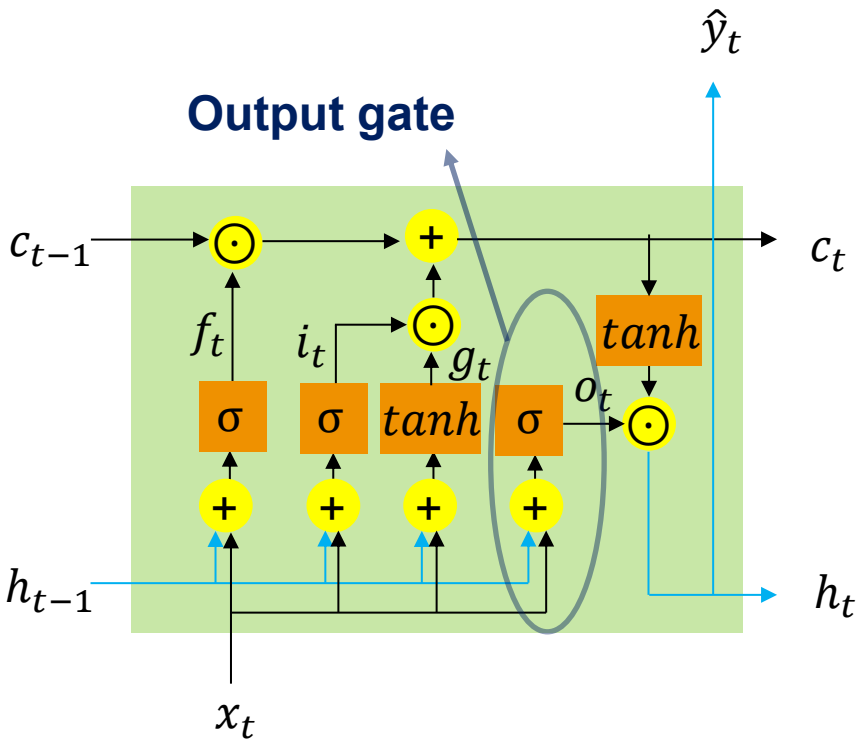
Input node – normal input as in standard RNN:

$$g_t = tanh(W_{ig}x_t + W_{hg}h_{t-1})$$

© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input node – normal input as in standard RNN:

$$g_t = tanh(W_{ig}x_t + W_{hg}h_{t-1})$$

Cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

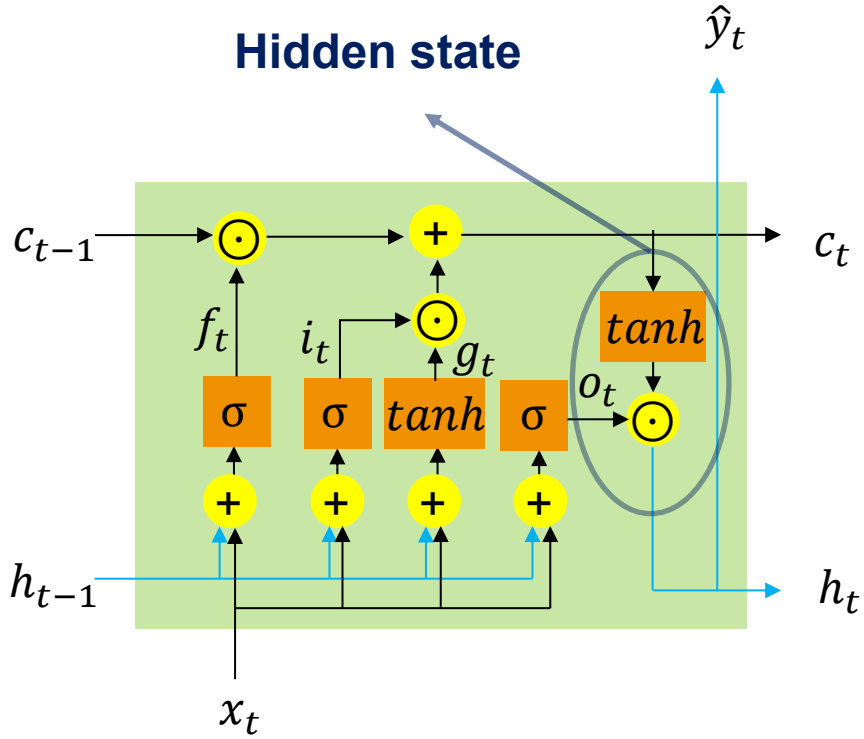
Output gate controls how much the cell state influences the output:

$$o_t = \sigma(W_{io}x_t + W_{ho}h_{t-1})$$

© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input node – normal input as in standard RNN:

$$g_t = tanh(W_{ig}x_t + W_{hg}h_{t-1})$$

Cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

Cell output affected by cell internal state

Output gate controls whether the memory cell influences the current output.:

$$o_t = \sigma(W_{io}x_t + W_{ho}h_{t-1})$$

Hidden state:

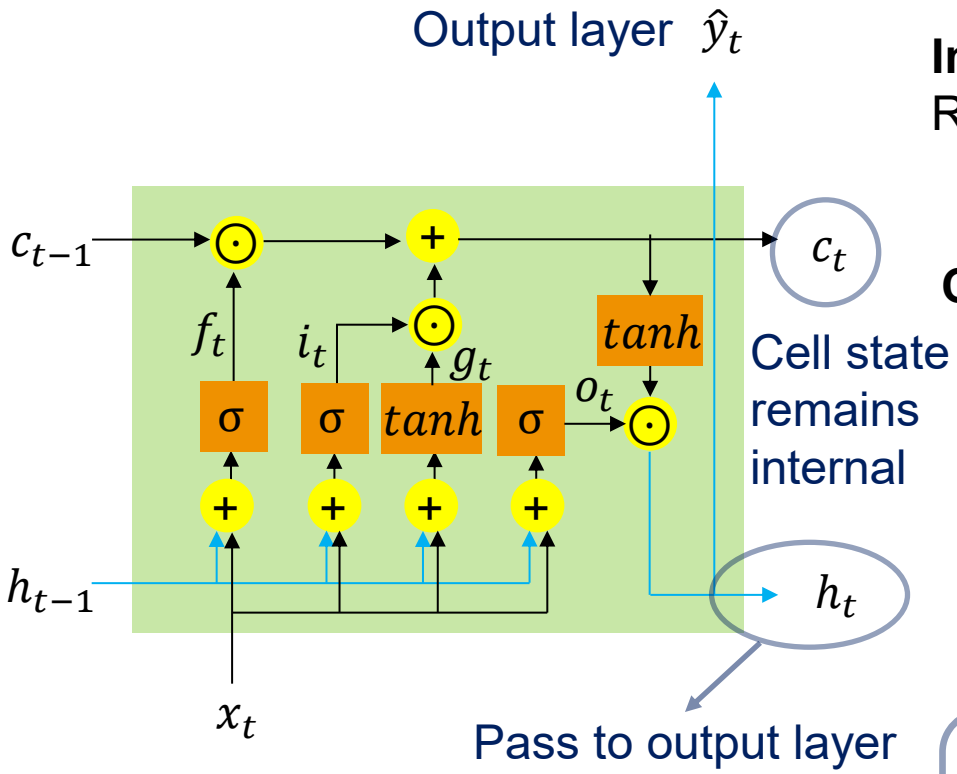
$$h_t = o_t \odot \tanh(c_t)$$

Output of a LSTM memory cell

© Xuhui Huang

Long Short-Term Memory (LSTM)

LSTM uses gating mechanism to mitigate gradient vanishing/exploding:



Input node – normal input as in standard RNN:

$$g_t = tanh(W_{ig}x_t + W_{hg}h_{t-1})$$

Cell state:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

Cell output affected by cell internal state

Hidden state (output):

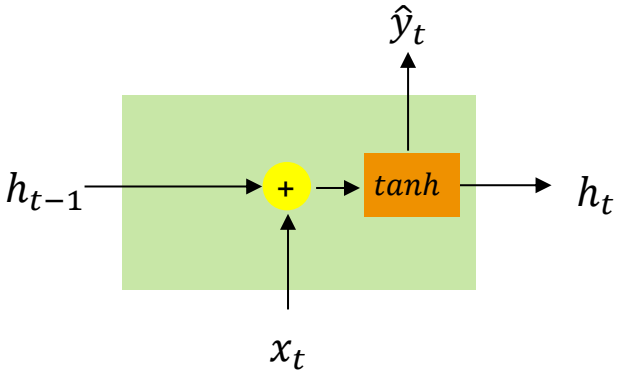
$$h_t = o_t \odot \tanh(c_t)$$

Only h_t is passed to the output layer, while the memory state c_t stays internal.

© Xuhui Huang

LSTM vs. Standard RNN

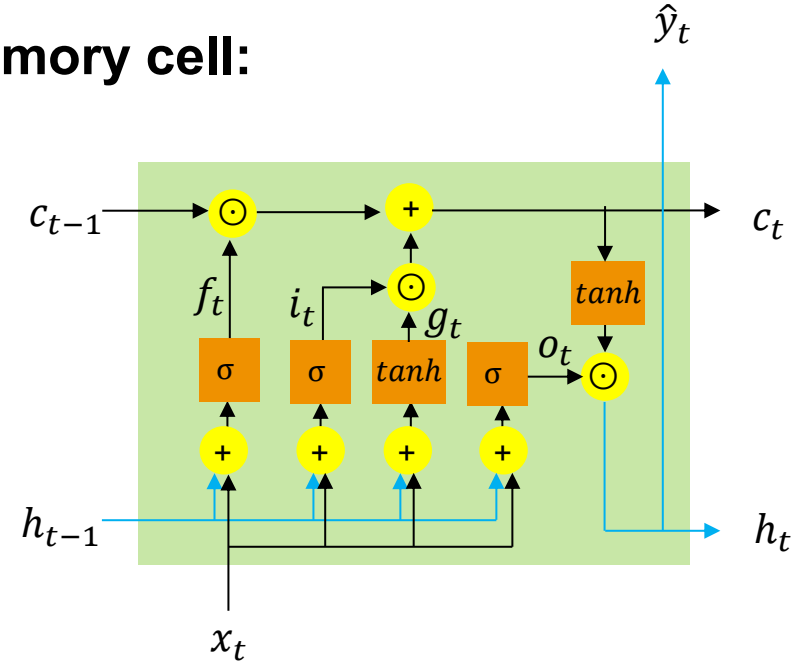
Standard RNN cell:



Learn only long-term memory:

Backpropagation through time learns long-term dependencies of weights.

LSTM memory cell:



Learn both long- and short-term memory via the regulations of input, forget and output gates:

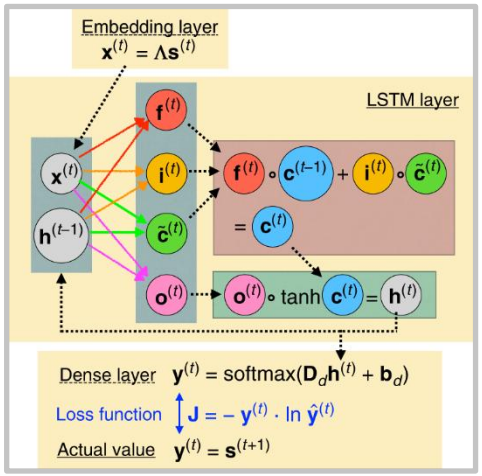
<u>Long term memory</u>: The weights change slowly during training, encoding general knowledge about the data.

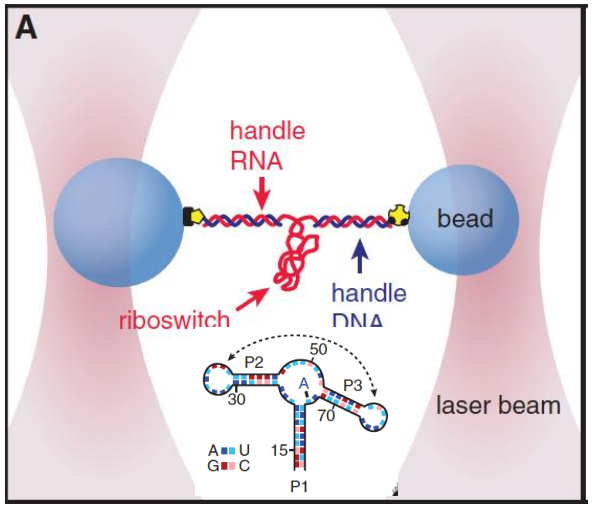
<u>Short-term memory</u> in the form of short-lived activations, which pass from each node to successive nodes.

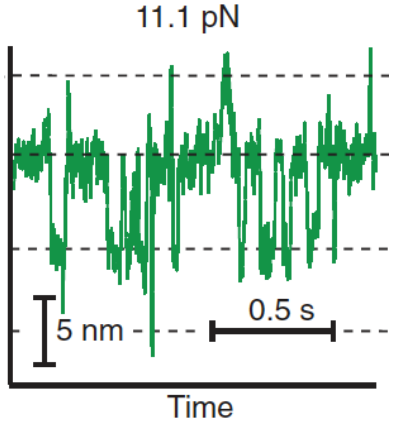
© Xuhui Huang

Learning Single-molecular Kinetics by LSTM

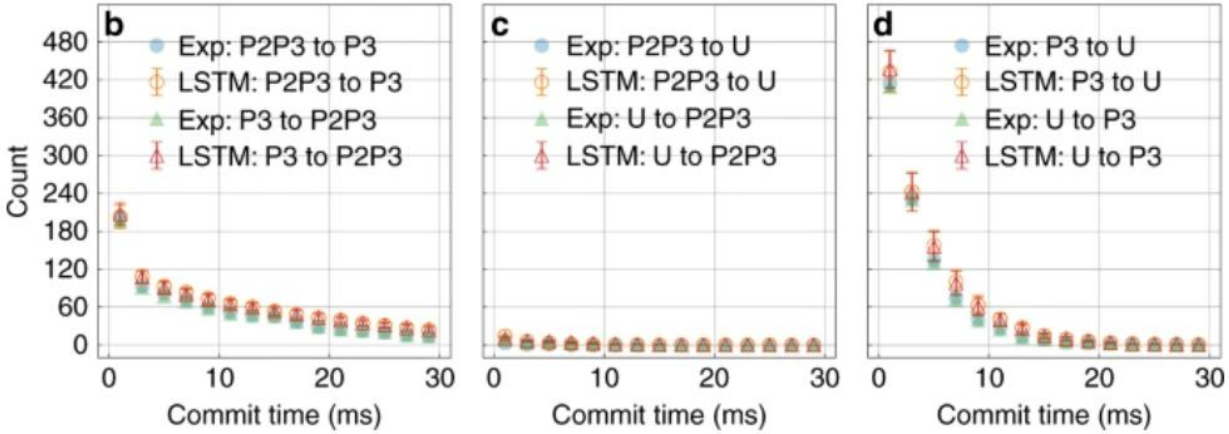
Training a LSTM model to learn single-molecular kinetics.







Exp single-molecular traj.



LSTM predicted kinetics agree well with single-molecular exp for Riboswitch folding.

Nat Comm, 11, 5115, (2020)

© Xuhui Huang

Exercise

Which of the following statements about LSTM gates are true?

(A). The forget gate in an LSTM controls how much past information should be discarded from the cell state.

(B). The input gate determines how much new information should be added to the cell state.

(C). The output gate directly updates the cell state based on the current input and previous hidden state.

(D). The forget, input, and output gates work together to regulate the flow of information, helping LSTMs mitigate the vanishing gradient problem.

© Xuhui Huang

Exercise

Which of the following statements about LSTM gates are true?

(A). The forget gate in an LSTM controls how much past information should be discarded from the cell state. - True

(B). The input gate determines how much new information should be added to the cell state. - True

(C). The output gate directly updates the cell state based on the current input and previous hidden state. – False

The output gate does not directly update the cell state

(D). The forget, input, and output gates work together to regulate the flow of information, helping LSTMs mitigate the vanishing gradient problem. - True

© Xuhui Huang

Exercise

What is the purpose of the **sigmoid activation function** in the gates of an LSTM?

(A). To introduce non-linearity and output values between -1 and 1.

(B). To output binary values (0 or 1) for gate operations.

(C). To output values between 0 and 1, representing the fraction of information to retain or discard.

(D). To compute the candidate cell state.

© Xuhui Huang

Exercise

What is the purpose of the sigmoid activation function in the gates of an LSTM?

(A). To introduce non-linearity and output values between -1 and 1.

(B). To output binary values (0 or 1) for gate operations.

(C). To output values between 0 and 1, representing the fraction of information to retain or discard. - True

(D). To compute the candidate cell state.

© Xuhui Huang

Exercise

Which of the following statements about LSTM are true?

(A). LSTM models are more stable in training compared to standard RNNs.

(B). LSTM considers both long- and short-term dependencies using memory cells.

(C). LSTM can explicitly trace memory over time.

(D). The gating mechanism in LSTM helps mitigate the vanishing gradient problem in training.

© Xuhui Huang

Exercise

Which of the following statements about LSTM are true?

(A). LSTM models are more stable in training compared to standard RNNs. -True

(B). LSTM considers both long- and short-term dependencies using memory cells. -True

(C). LSTM can explicitly trace memory over time.

- False: *While LSTM maintains memory, it does not explicitly store and retrieve specific past values. Instead, memory is controlled implicitly through learned gating mechanisms.*

(D). The gating mechanism in LSTM helps mitigate the vanishing gradient problem in training. -True

© Xuhui Huang

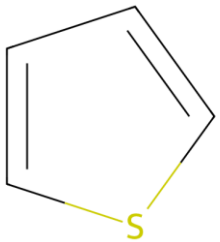
Outline

- Recurrent neural networks (RNNs)
  - Forward prorogation
  - Backpropagation
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - Long short-term memory (LSTM) model
- **Optimizing RNN training for better predictions**
- Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.

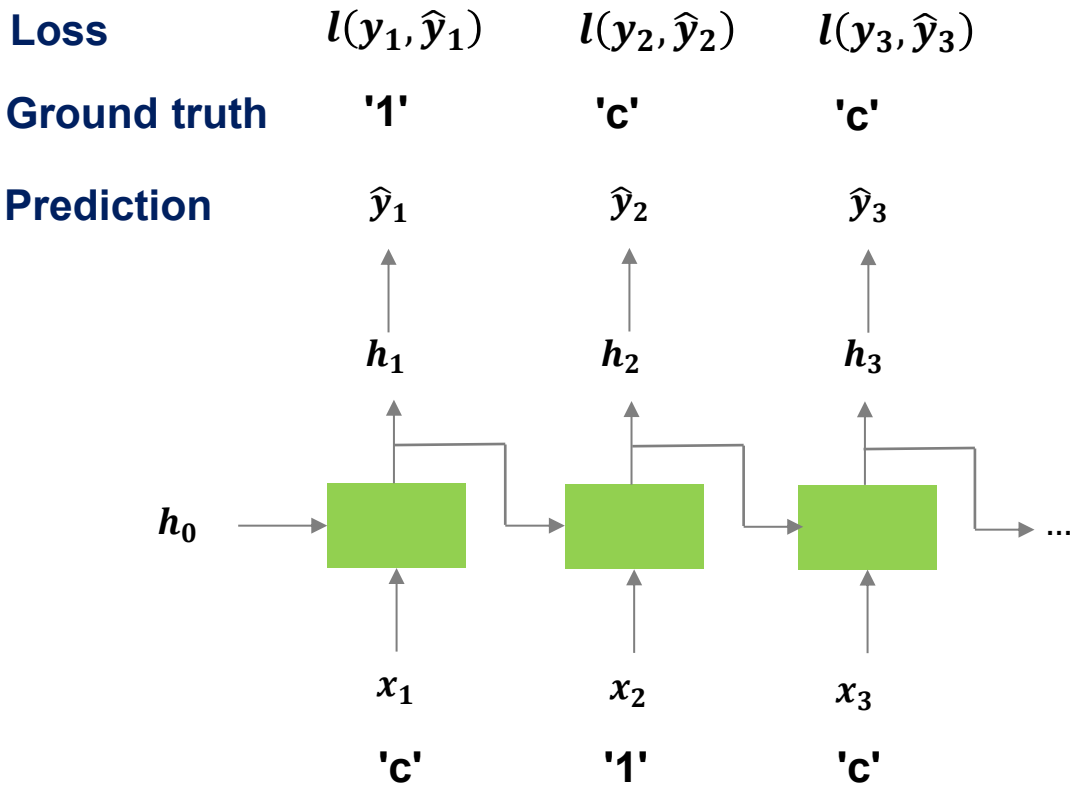
© Xuhui Huang

What is Teacher Forcing?

Teacher Forcing is a RNN training strategy where the model uses the actual ground-truth output from the previous time step as the input for the next step.



c1cccs1

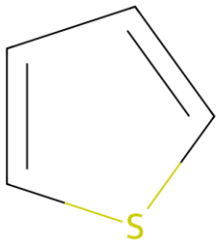


Speeds up training by providing correct context and helps the model learn better at the beginning of training.

© Xuhui Huang

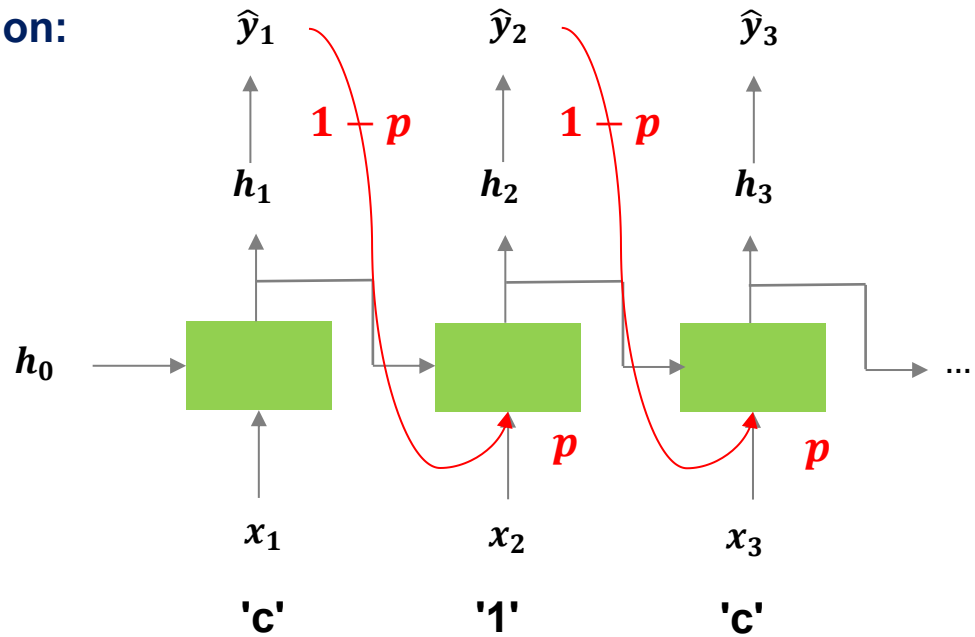
What is Free Running?

Free Running: The model uses its own predicted output at each time step instead of the ground truth.



c1cccs1

Prediction:



- Helps the model adapt to real-world scenarios where true outputs are unavailable.
- Prevents exposure bias (model relying too much on perfect training data).

© Xuhui Huang

How to Balance Both? – Scheduled Sampling

Training Method	Pros	Cons
Teacher Forcing	Faster training, prevents errors from propagating	Leads to **exposure bias**, struggles with real-world predictions
Free Running	More realistic, helps generalization	Harder to train, prone to error accumulation

Scheduled Sampling: a strategy to mix the two training methods**.**

- Early training: Use ground-truth tokens (teacher-forcing) to accelerate training, since early predictions are unreliable.

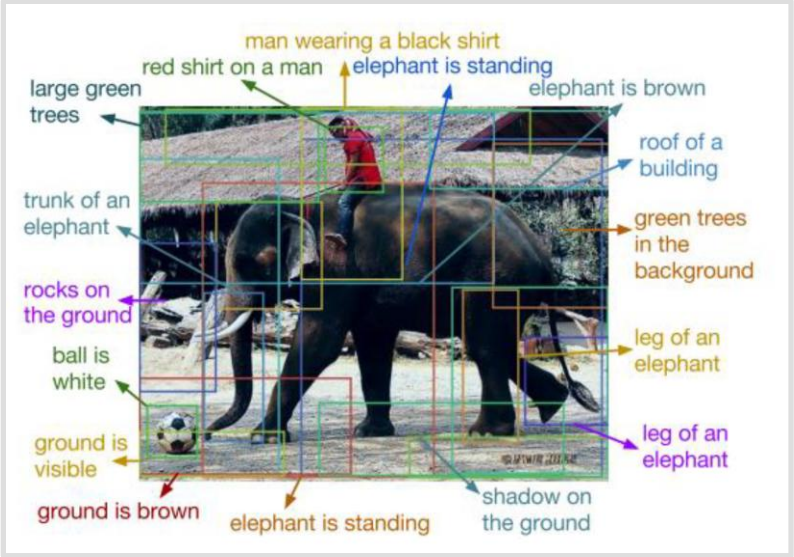
- Mid training: Gradually mix ground-truth and model predictions to improve adaptation.

- Late Training: Rely entirely on model-generated predictions to ensure it performs well on real-world data and doesn't rely too much on perfect training conditions.

© Xuhui Huang

How to Balance Both? – Scheduled Sampling

Task: Image captioning. Input: one image, Output: sequence of words



https://doi.org/10.48550/arXiv.1511.07571

Various metrics (the higher the better) for the image captioning task.

Approach vs Metric	BLEU-4	METEOR	CIDER
Baseline	28.8	24.2	89.5
Baseline with Dropout	28.1	23.9	87.0
Always Sampling	11.2	15.7	49.7
Scheduled Sampling	**30.6**	**24.3**	**92.1**
Uniform Scheduled Sampling	29.2	24.2	90.9
Baseline ensemble of 10	30.7	25.1	95.7
Scheduled Sampling ensemble of 5	**32.3**	**25.4**	**98.7**

Scheduled sampling improves RNN training for sequence generation tasks like machine translation, image captioning, and SMILES generation.

Bengio, et al., (2015). ***arXiv***. https://doi.org/10.48550/ARXIV.1506.03099

© Xuhui Huang

Outline

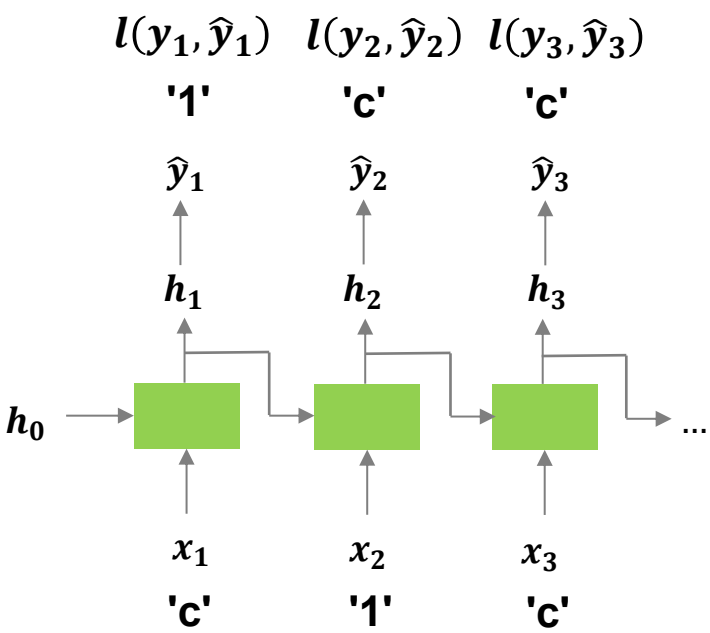
- Recurrent neural networks (RNNs)
  - Forward prorogation
  - Backpropagation
- Gradient Vanishing/Exploding
  - Gradient Clipping
  - Long short-term memory (LSTM) model
- Optimizing RNN training for better predictions
- **Building a RNN to predict Molecular solubility– Hands-on implementation for a real chemistry problem.**

© Xuhui Huang

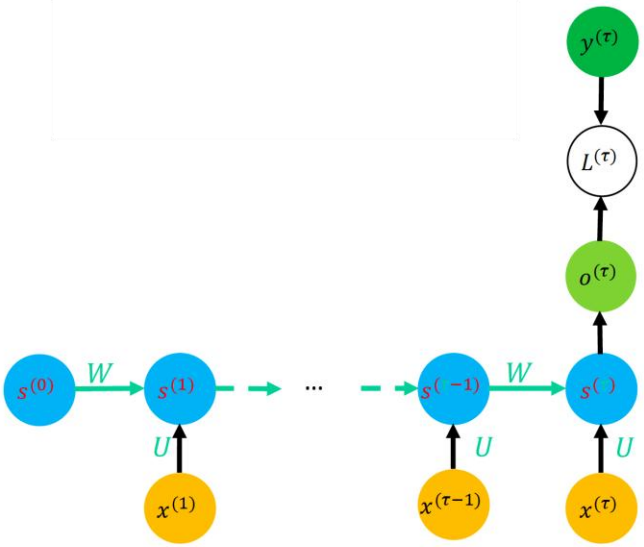
Building a RNN to Predict Molecular Solubility

Task: Training a RNN model to predict experimentally measured solubility.

Reference_ch6_Part_3_rnn_colab.ipynb



RNNs that use the output at previous steps.



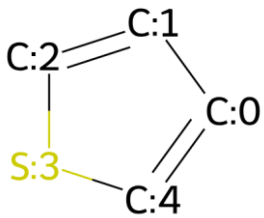
RNNs that only output at the end.

We use this RNN variant to predict the solubility as output at the end!

© Xuhui Huang

SMILEs Strings as Sequential Text

Training data: 1128 (90/10 split) compounds with experimentally measured solubility



logS = -1.33

Vocabulary *Vocab size* = 46

["PAD", "UNK", "B", "Br", "Cl", "C", "H", "N", "O", "S", "P", "F", "I“, "b", "c", "n", "o", "s", "p", "[", "]", "(", ")", ".", " ", "=", "#“, ...]

Each element in the vocabulary is called a token. "UNK" is reserved for tokens out of vocabulary (OOV), "PAD" is reserved for batching. Each token has its unique integer identifier.

Valid SMILES	Tokenization
c1ccsc1	[14, 37, 14, 14, 17, 14, 37]
c1sccc1	[14, 37, 17, 14, 14, 14, 37]
s1cccc1	[17, 37, 14, 14, 14, 14, 37]
c1cccs1	[14, 37, 14, 14, 14, 17, 37]

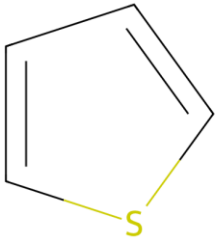
Data augmentation makes our model robust to predict for SMILES generated by arbitrary rules

Delaney Dataset. ***J. Chem. Inf. Comput. Sci.*** 44, 1000, (2004)

© Xuhui Huang

Padding and Batching

After padding, sentences of different lengths can be processed simultaneously in the same batch.



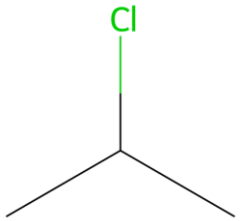
logS = -1.33

c1ccsc1

Tokens: ['c', '1', 'c', 'c', 'c', 's', '1']

Number of tokens: 7

Token ids: [14, 37, 14, 14, 17, 14, 37]



logS = -1.41

ClC(C)C

Tokens: ['Cl', 'C', '(', 'C', ')', 'C']

Number of tokens: 6

Token ids: [4, 5, 21, 5, 22, 5]

Token ids after padding: [0, 4, 5, 21, 5, 22, 5]

Padding (Length: 7)

Here we choose pre-padding zeros (padding at the beginning of the sentence) to make the length of token ids aligned to the longest sentence in this batch.

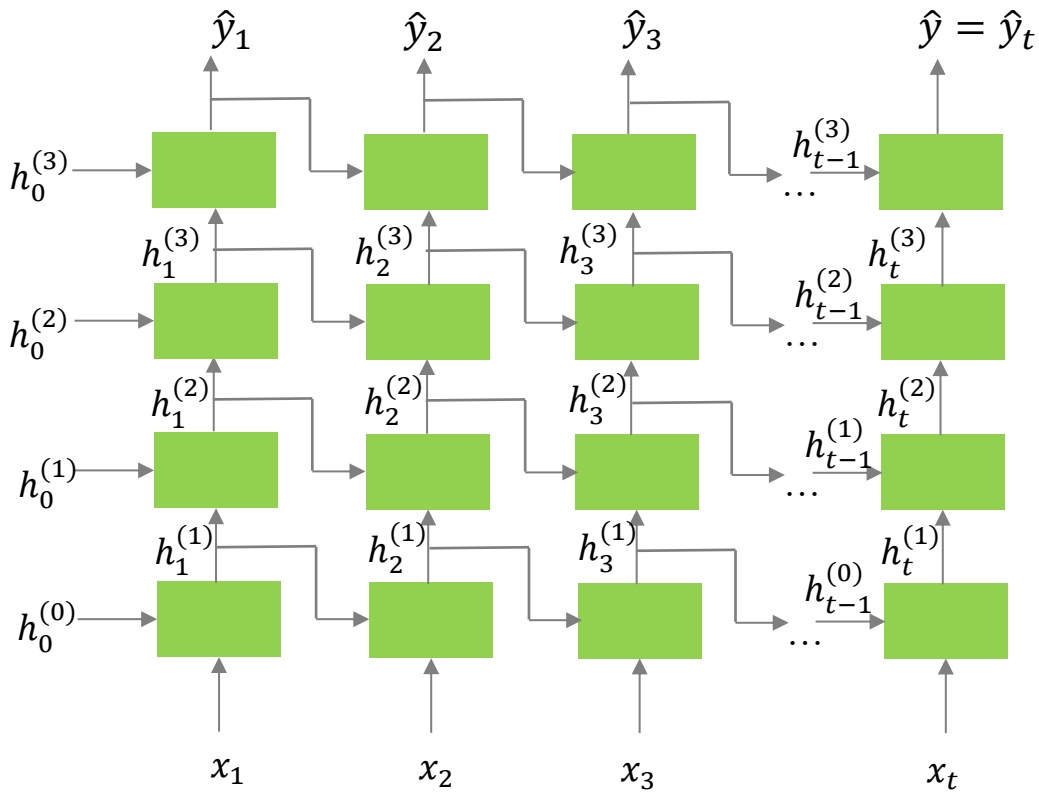
Other common practices include appending “EOS” (end-of-sentence) tokens, and post-padding + masking.

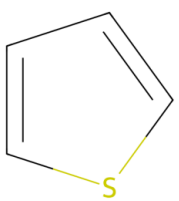
© Xuhui Huang

RNN Architecture

Only the last output contribute to the loss function MSE $= \frac{1}{n}\sum_{j=1}^{n}\left(\hat{y}^{(j)} - y^{(j)}\right)^2$

4-Layer RNN





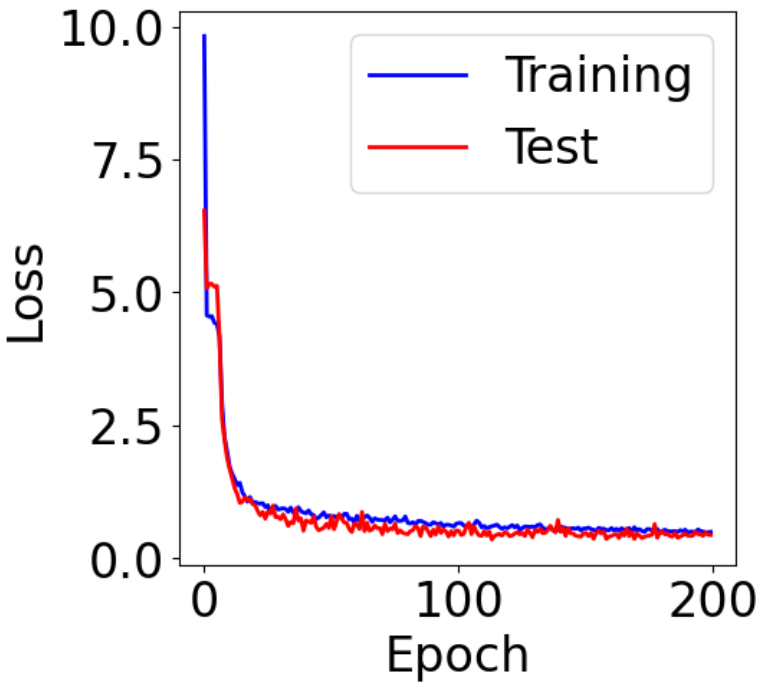
c1ccsc1

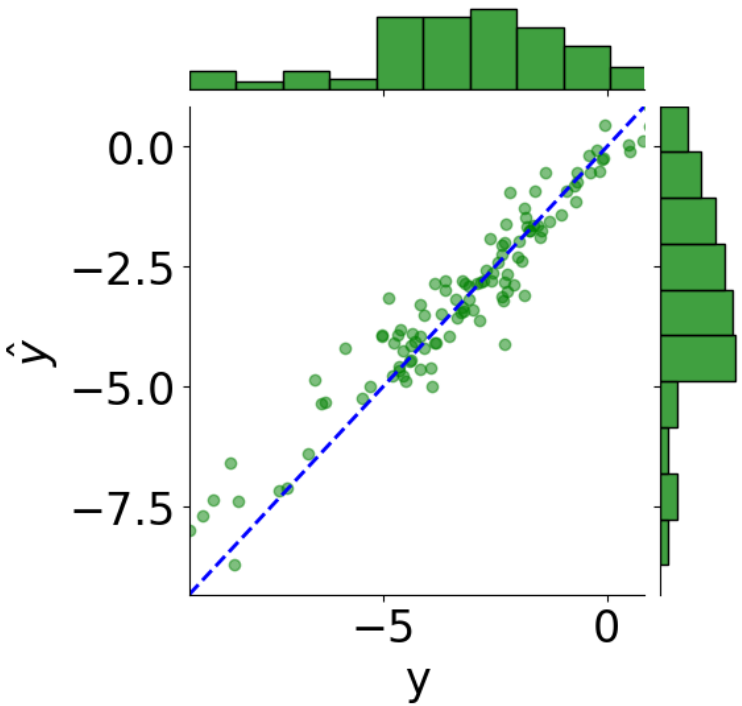
logS = -1.33

Embedding				…	
Token ID	14	37	14	…	37
Token	'c'	'1'	'c'	…	'1'

© Xuhui Huang

Training Evaluation





Evaluation metrics:

MSE = 0.42

R^2 = 0.92

© Xuhui Huang